

Distributed Tracing and Logging

You should understand these as two related but different observability mechanisms:

```
Logging
→ "What happened inside this service?"

Distributed tracing
→ "How did one request travel across multiple services?"
```

They become especially important in **microservices**, where one user request may cross several services.

1. Why Do We Need Them?

Consider a request:

```
Client
↓
API Gateway
↓
Order Service
↓
Payment Service
↓
External Payment Provider
↓
History Service
```

The customer says:

```
"My payment failed."
```

Where did it fail?

Possibilities:

```
Gateway
Order Service
Payment Service
Database
External provider
History Service
Network
```

A single-service log is not enough.

We need:

```
Logs
+
Metrics
+
Distributed Traces
```

Together they are part of **observability**.

2. Logging

Logging records important events that occur inside an application.

Example:

```
log.info("Processing payment: transactionId={}", transactionId);
```

If something fails:

```
log.error(  
    "Payment processing failed: transactionId={}",  
    transactionId,  
    exception  
);
```

A log entry may contain:

```
timestamp  
log level  
service name  
thread  
message  
transaction ID  
trace ID  
additional fields
```

3. Log Levels

You should know these:

```
TRACE  
DEBUG  
INFO  
WARN  
ERROR
```

TRACE

Very detailed diagnostic information.

Usually disabled in production unless specifically investigating something.

DEBUG

Detailed developer-level diagnostic information.

INFO

Normal important application events.

Example:

```
Payment processing started  
Payment completed  
Batch started
```

WARN

Something unusual happened, but the application may continue.

Example:

```
Downstream response slower than expected
Retrying request
```

ERROR

An operation failed.

Example:

```
Payment provider timeout
Database exception
```

4. What Should You Log?

Good logging:

```
Transaction ID
Business operation
Status
Relevant response code
Duration
Error type
Trace ID
```

Example:

```
transactionId=TX123
operation=PAYMENT
status=FAILED
provider=HYDRA
durationMs=3200
traceId=abc123
```

Avoid logging sensitive information.

For a payment application, be especially careful with:

```
passwords
tokens
secrets
full authentication headers
card/payment-sensitive data
```

You should **mask or omit sensitive fields**.

5. Structured Logging

Instead of:

```
"Payment TX123 failed"
```

you can use structured fields:

```
{
  "level": "ERROR",
  "service": "payment-service",
  "transactionId": "TX123",
  "provider": "HYDRA",
  "status": "FAILED",
  "traceId": "abc123",
  "durationMs": 3200
}
```

Why is this better?

Log management tools can search fields directly:

```
transactionId = TX123
provider = HYDRA
status = FAILED
```

This is much better for production debugging than parsing arbitrary log text.

6. Logging in Spring Boot

Typically:

```
private static final Logger log =
    LoggerFactory.getLogger(PaymentService.class);
```

Then:

```
log.info(
    "Processing payment transactionId={}",
    transactionId
);
```

For errors:

```
log.error(
    "Payment failed transactionId={}",
    transactionId,
    ex
);
```

Use parameterized logging rather than string concatenation:

```
// Prefer
log.info("Payment id={}", id);

// Avoid
log.info("Payment id=" + id);
```

7. Correlation ID

Suppose a request enters your system:

```
Correlation-ID: ABC123
```

You propagate it through downstream calls:

```
Gateway
  ↓ ABC123
Service A
  ↓ ABC123
Service B
  ↓ ABC123
Service C
```

Now all logs contain:

```
ABC123
```

You can search for:

```
ABC123
```

and reconstruct the request's journey.

A correlation ID is extremely useful even without full distributed tracing.

8. Distributed Tracing

Distributed tracing goes one level further.

It tracks a request across services as a **trace** made up of **spans**.

Example:

```
Trace
|
├─ Gateway span
├─ Order Service span
├─ Payment Service span
├─ External API span
└─ History Service span
```

9. Trace vs Span

These two terms are essential.

Trace

Represents the entire request/workflow.

Example:

```
Trace ID = ABC123
```

Span

Represents one operation within that trace.

```
Span 1 → Gateway
Span 2 → Payment Service
Span 3 → Hydra call
Span 4 → DB query
```

Conceptually:

```
Trace: ABC123

Gateway      [-----]
Payment      [-----]
Hydra        [----]
DB           [--]
```

10. Why Tracing Is Better Than Just Logs

Suppose your request takes **8 seconds**.

Logs may tell you:

```
Payment started
Hydra called
Payment completed
```

But tracing can tell you:

```
Gateway      20ms
Payment      120ms
Hydra        7,500ms
Database      80ms
```

Now you've immediately identified:

Hydra consumed most of the latency.

Tracing is therefore extremely useful for **distributed performance analysis**.

11. Trace ID and Span ID

A trace typically has:

```
Trace ID
Span ID
Parent Span ID
```

Example:

```
Trace ID = abc123

Gateway:
Span ID = 001

Payment:
Span ID = 002
Parent = 001
```

```
Hydra:
Span ID = 003
Parent = 002
```

This creates a hierarchy:

```
001 Gateway
  ↓
002 Payment
  ↓
003 Hydra
```

12. How Trace Context Travels

When Service A calls Service B, tracing information is propagated through HTTP headers.

Modern distributed tracing commonly uses **W3C Trace Context** headers such as:

```
traceparent: ...
```

Conceptually:

```
Service A
  ↓
HTTP request
  ↓
trace context
  ↓
Service B
```

Service B extracts the context and continues the same trace.

13. OpenTelemetry

For modern systems, **OpenTelemetry (OTel)** is an important technology to know.

It provides APIs/SDKs/instrumentation for:

- traces
- metrics
- logs

It is designed to provide vendor-neutral telemetry collection and export.

A simplified architecture:

```
Application
  ↓
OpenTelemetry instrumentation
  ↓
Telemetry
  ↓
Collector
  ↓
Backend
```

Possible backends include:

- Jaeger
- Grafana Tempo
- commercial observability platforms

14. OpenTelemetry vs Jaeger

Don't confuse these.

OpenTelemetry

Instrumentation/telemetry framework and standard ecosystem.

Jaeger

A distributed tracing backend/UI.

Conceptually:

```
Application
  ↓
OpenTelemetry
  ↓
Jaeger
```

15. Spring Boot + Micrometer

In modern Spring applications, **Micrometer** is commonly used for observability/metrics integration.

Spring Boot's observability ecosystem can integrate traces and metrics with OpenTelemetry depending on your configuration.

You don't need to memorize all configuration properties, but understand the ecosystem:

```
Spring Boot
  ↓
Micrometer / OpenTelemetry
  ↓
Telemetry backend
```

16. Logging vs Tracing

This distinction is highly likely to be asked.

Logging	Distributed Tracing
Records events	Tracks request flow
Usually text/structured records	Trace + spans
Good for detailed application information	Good for cross-service flow
Great for errors/debugging	Great for latency/dependency analysis

Can be searched by IDs	Visual request timeline
Service-focused	System-wide

They complement each other.

17. Best Practice: Put Trace ID in Logs

This is where they become powerful together.

Example log:

```
{
  "level": "ERROR",
  "service": "payment-service",
  "traceId": "abc123",
  "spanId": "span789",
  "transactionId": "TX1001",
  "message": "Payment provider timeout"
}
```

Then:

```
Trace
abc123
↓
See all spans
↓
Find payment-service span
↓
Search logs using abc123
↓
Get detailed error
```

This is a strong observability workflow.

18. Logging vs Audit Logging

Don't confuse application logs with audit logs.

Application log

Used mainly for:

```
debugging
operations
diagnostics
errors
```

Audit log

Records important business/security actions.

Example:

```
user=admin
action=PAYMENT_REVERSAL
```

```
transactionId=TX123
timestamp=...
```

Audit records often have stronger retention/integrity requirements than ordinary application logs. For payment systems, auditability is particularly important.

19. Logging vs Metrics vs Tracing

You should know this three-way distinction.

Logs

What happened?

Example:

```
Hydra call failed
```

Metrics

How often/how much?

Example:

```
payment_failure_rate = 2.3%
```

Traces

Where did this request spend time/fail?

Example:

```
Gateway → BPS → Hydra
              ↑
            7.5 sec
```

Together:

```
Observability
├─ Logs
├─ Metrics
└─ Traces
```

20. Production Scenario

Suppose users report:

“Payments are taking 10 seconds.”

Step 1 — Metrics

Check:

```
Payment latency
Error rate
Request throughput
```

Suppose:

```
p95 latency = 10 sec
```

Step 2 — Trace

Open a slow trace:

```
Gateway    20ms
BPS        100ms
Hydra      9.5sec
BHS        50ms
```

Now you know the likely bottleneck is Hydra.

Step 3 — Logs

Search using:

```
traceId
transactionId
```

You find:

```
Hydra read timeout
```

Now you have:

```
Metrics → detected problem
Trace → located problem
Logs → identified detailed cause
```

That is observability in practice.

21. Another Production Scenario

```
"One particular payment failed. Find out why."
```

Use:

```
Transaction ID
↓
Search logs
↓
Find trace ID
↓
Open trace
↓
Follow service calls
```

```
↓  
Find failing span  
↓  
Inspect corresponding logs
```

This is an excellent operational workflow to understand.

22. What Is Log Aggregation?

In a microservice environment:

```
Service A → logs  
Service B → logs  
Service C → logs  
Service D → logs
```

You don't want to log into four servers individually.

A centralized logging platform collects them:

```
Services  
↓  
Log collector  
↓  
Centralized logging  
↓  
Search / dashboard
```

Examples:

```
ELK / Elastic Stack  
Splunk  
Loki
```

Since you've mentioned **Splunk** in your work background, this is particularly relevant to understand conceptually, but the general principles are the same across platforms.

23. Log Correlation

Imagine:

```
Transaction ID = TX123  
Trace ID = abc123
```

Search:

```
transactionId=TX123
```

and find:

```
Gateway  
BPS  
Hydra  
BHS
```

That gives you the end-to-end story.

This is why identifiers should be consistently propagated.

24. Important Logging Best Practices

1. Don't log sensitive data

Avoid:

```
password
JWT
API keys
secrets
sensitive payment data
```

2. Use structured logs

Prefer fields over huge free-form strings.

3. Include identifiers

Useful examples:

```
transactionId
requestId
traceId
user/service identifier where appropriate
```

4. Don't log everything at ERROR

Incorrect:

```
log.error("Entering payment method");
```

Use appropriate levels.

5. Don't log entire request/response blindly

Especially dangerous for payment/security data.

6. Include exception details when useful

```
log.error("Payment failed txId={}", txId, ex);
```

25. Sampling in Distributed Tracing

Tracing every request can generate a large volume of telemetry.

So systems may use **sampling**.

For example:

```
1,000,000 requests
```

↓
trace 10% or selected requests

Sampling can be:

- head-based
- tail-based

At a basic interview level, understand:

Sampling controls how much trace data is retained/processed.

For troubleshooting critical payment operations, organizations may choose policies that preserve more relevant traces.

26. Logs Can Also Be Correlated Without Full Tracing

Suppose your application generates:

X-Correlation-ID: ABC123

and passes it through every service.

Then:

Gateway → ABC123
BPS → ABC123
Hydra integration → ABC123
BHS → ABC123

You can reconstruct the request manually.

This is useful, but full distributed tracing gives you richer information:

service
span
parent-child relationships
duration
dependencies

27. Tracing an Asynchronous Message

Tracing isn't limited to REST.

Suppose:

Service A
↓
Kafka
↓
Service B

Trace context can be propagated through message headers/metadata.

Conceptually:

```
Trace ABC123
  ↓
Producer span
  ↓
Kafka
  ↓
Consumer span
```

This allows you to trace asynchronous workflows too.

28. Common Interview Questions

Logging

1. What is logging?
2. What log levels do you know?
3. INFO vs DEBUG vs ERROR?
4. What is structured logging?
5. What should you avoid logging?
6. What is centralized logging?
7. How would you debug a production issue from logs?

Distributed tracing

8. What is distributed tracing?
9. Trace vs span?
10. Trace ID vs Span ID?
11. How does trace context propagate?
12. What is correlation ID?
13. Why isn't logging alone enough?
14. What is OpenTelemetry?
15. OpenTelemetry vs Jaeger?
16. What is trace sampling?

Observability

17. Logs vs metrics vs traces?
18. How would you diagnose high latency?
19. How would you follow one transaction across multiple services?
20. How would you find which downstream service caused an issue?

29. 3-Year-Level Scenario Question

Interviewer:

"A request goes through Gateway → Order Service → Payment Service → external payment provider. The request takes 8 seconds. How would you find where the latency comes from?"

Strong answer:

“I’d first look at service-level latency metrics to confirm the issue and identify whether it’s widespread or isolated. Then I’d inspect a distributed trace for a slow request using the trace ID. The trace shows the duration of each span and downstream call, so I can identify where most of the 8 seconds is being spent. After locating the slow span, I’d correlate its trace ID and transaction/request ID with centralized logs to inspect the detailed error, timeout or downstream response. If the issue is related to the external provider, I’d then check its response times and error/timeout metrics.”

That’s a good **3-year backend answer**.

30. BPS Questions — Investigate Your Codebase

Now apply this to your actual work:

1. What identifier do you use to trace a payment request?
2. Does BPS generate or receive a correlation/request ID?
3. Is that ID propagated to downstream services?
4. What identifiers are present in BPS logs?
5. Can you search one payment across BPS, Hydra/Chase and BHS?
6. Does BPS use structured logging?
7. Which log levels are used for payment processing?
8. How are vendor failures logged?
9. How do you distinguish business failure from technical failure in logs?
10. Can you correlate a production issue in Splunk with a specific transaction?
11. Is distributed tracing implemented?
12. Which tracing/observability technology is used?
13. What metrics exist for payment latency/failure?
14. How would you diagnose a slow Hydra/Chase call?
15. How would you trace a recurring-payment batch failure back to a specific chunk/transaction?

Interview-ready summary

“Logging records detailed application events, while distributed tracing tracks a request as it moves across multiple services. A trace represents the complete request and consists of spans representing individual operations or service calls. Trace IDs and correlation IDs allow us to connect logs and traces across services. In production, metrics can identify that there is a latency or error problem, tracing can show where in the distributed request it occurred, and logs provide the detailed diagnostic information needed to determine the root cause. OpenTelemetry is a modern vendor-neutral framework for collecting telemetry, while systems such as Jaeger can store and visualize traces. Good observability also requires structured logging, sensitive-data protection, appropriate log levels and careful trace/log propagation.”

Remember this:

→ What happened?

Metrics

→ How much/how often?

Traces

→ Where did this request go and where did it spend time?

And the most useful production-debugging chain:

Metric

↓

Find abnormal service/operation

↓

Trace ID

↓

Distributed trace

↓

Slow/failing span

↓

Correlated logs

↓

Root cause